

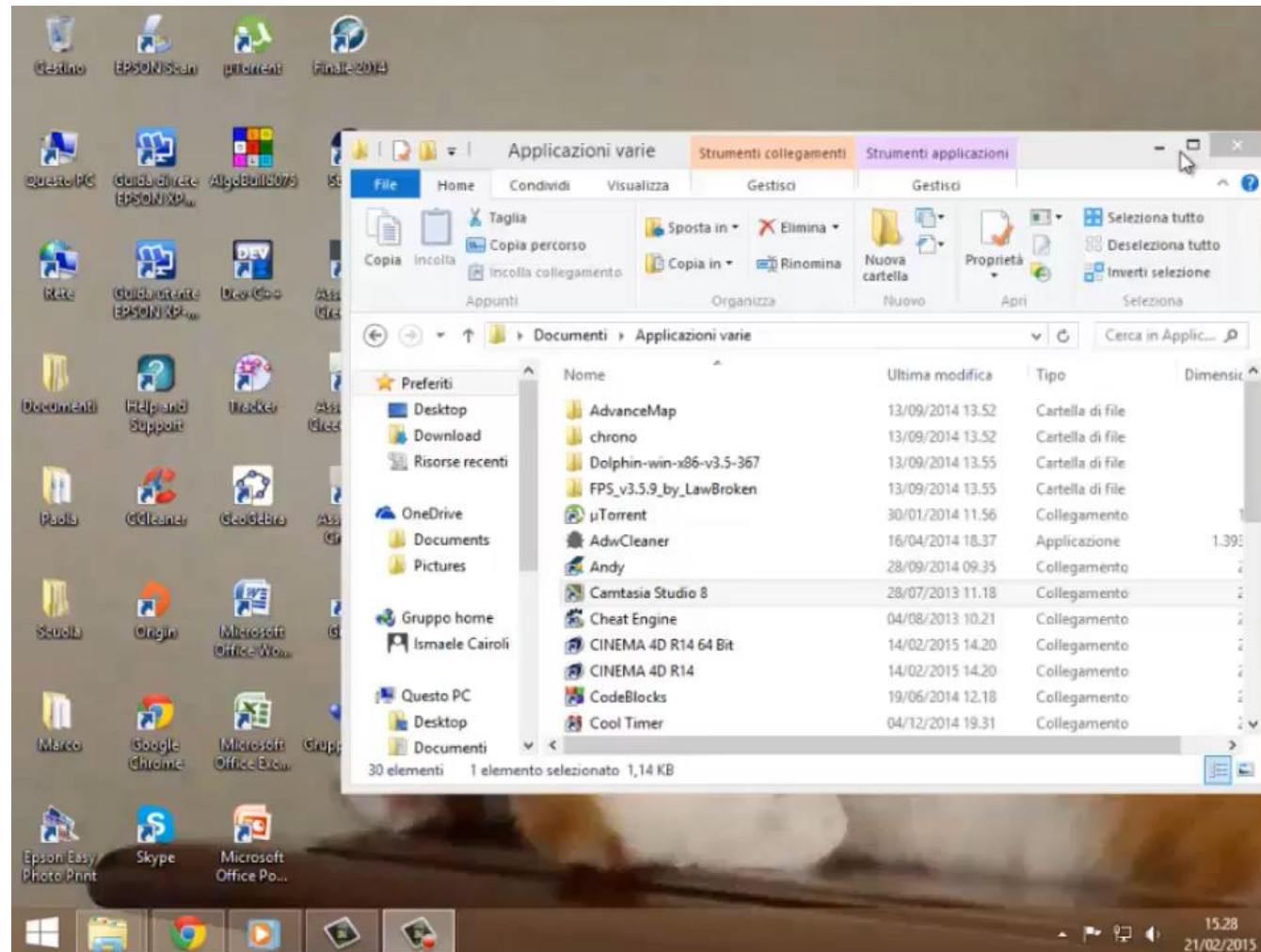
EXERCISE 4: IMAGE PROCESSING PART 1

Multimedia Database SS 26

Prof. Dr. Harald Kosch/ Prof. Dr. Mario Döller

Tutor: Manuel Lehner

Instructions vs. Picture



Source: Youtube
<https://youtu.be/rSf0FhtBML8>

1a) Image Formats

- **Vector Image**

- Makes use of sequential commands or mathematical statements which place lines or shapes in a 2-D or 3-D environment.
- Each control point has a definite (x,y) position and determines a graphic primitive, which may be shapes, curves, splines
- Occupies less space,
- Suitable for printing, since vector images scale well

- **Raster Image**

- Raster Images are pixel based
- Use bitmaps to store information
- Consumes more memory
- Scaling up leads to degradation.



1b) Image Formats: GIF

- **Color model:** RGB
- **Color depth:** 8 Bits per pixel – range of 24 Bits color space (256/16 M)
- **Lempel-Ziv-Welch (LZW) compression** – lossless
- Small file size
- Can represent animations, logos, etc..
- but not suitable for photos due to color limitation.



1b) Image Formats: PNG

- **Color model:** RGB/RGBA
- **Color depth:** variable – grey scale (1-16 Bpp) Indexed (1-8 Bpp 24 Bits Color palette), TrueColor (24/48 Bpp)
- **Compression:** Combination of LZSS and Huffman Coding (lossless)
- Doesn't support animation



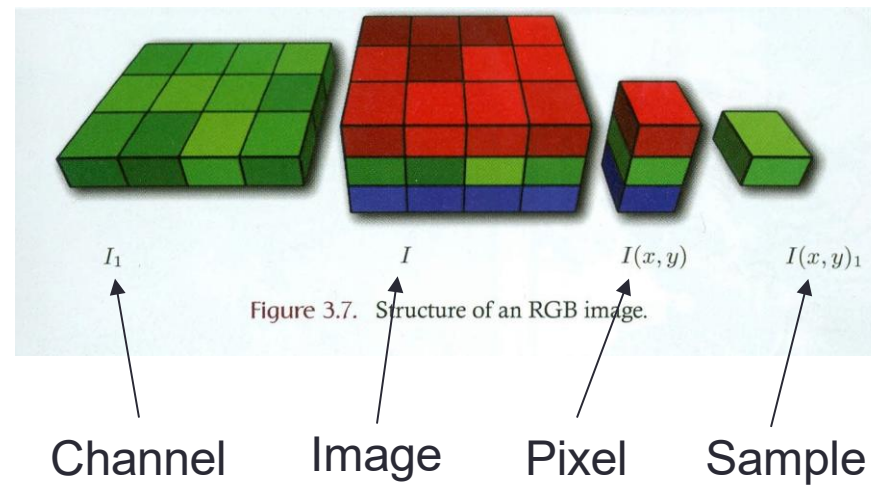
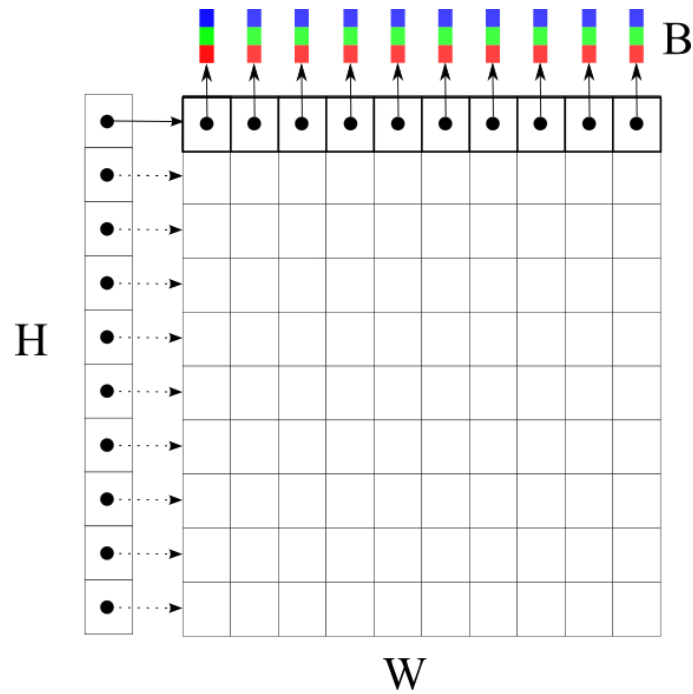
1b) Image Formats: JPEG

- **Color model:** RGB
- **Color depth:** 24 Bits per pixel (most common)
- **DCT-based compression**, lossy
- Doesn't support animation, transparency



2a) Image Processing

- Naive internal representation: 3-D Array
- `int[][][]` raster



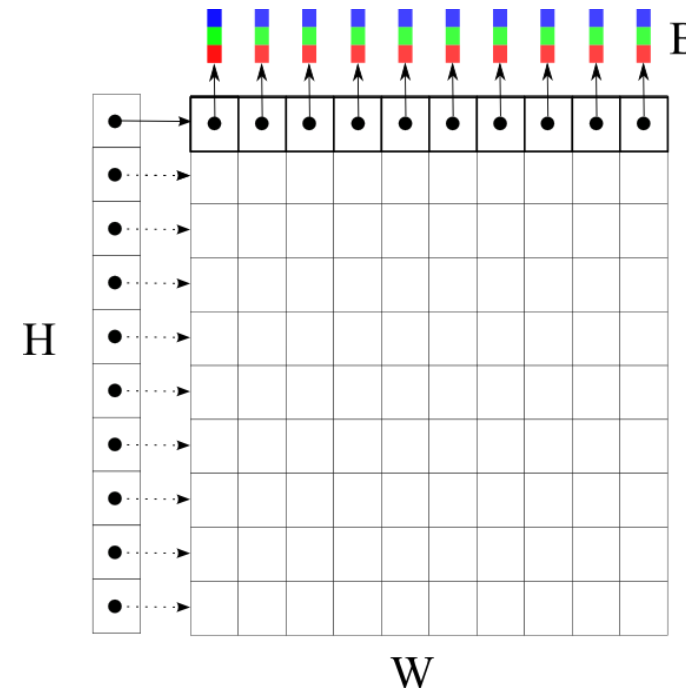
2) Image Processing

- **24bpp RGB image: 1 Byte = 8 bits per sample**
 - $\text{Size}(\text{raster}) = w \times h \times b$ samples (sample = channel value)
 - $\Rightarrow$ **Efficient implementation would require at most** $w \times h \times b$ Bytes of memory
- **Practically $\rightarrow$ more**
 - Java: multi-dimensional arrays = array of an array



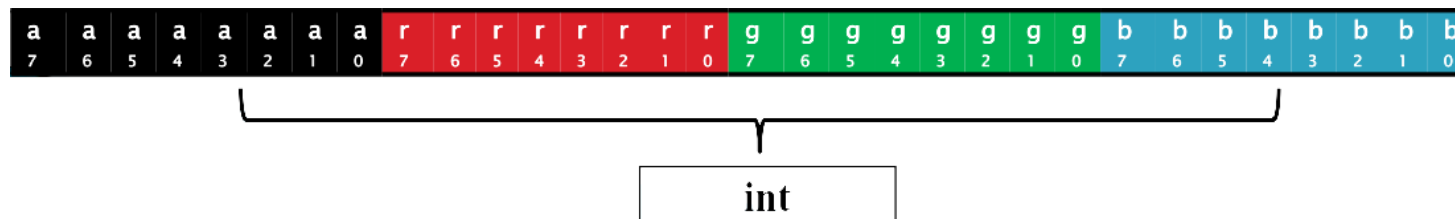
2a) Image Processing

- **Memory Consumption using 3-D raster array**
- **References (Pointers):**
 - $H+WH$ (4 Bytes for each)
- **Object Header (Array length attribute) :**
 - $1+H +WH$ (4 Bytes each)
- **Total amount of unnecessary memory consumed by this 3-D representation:**
 - $4(H+WH) + 4(WH + H +1)$
 $\sim 8 WH$
- **Image data**
 - $3WH$
- **Total ($W=H=12$)**
 - $4(H+WH) + 4(WH + H +1)+4(3WH) = 2980$ Bytes



2b) Image Processing

- Till now, a sample was stored in an integer value (**4-Bytes per sample**).
- However, most image processing applications **require a color depth** of at most **24 bits per pixel (1-Byte per sample)**.
- **Solution:** pack the entire pixel into a single int value.



2b) Image Processing

- **Memory Consumption using 2-D array**
- **References (Pointers) :**
 - H (4 Bytes for each)
- **Array length attribute:**
 - 1+H (4 Bytes each)
- **Total amount of unnecessary memory consumed**
 - $2H+1$
 - $\sim 2H$
- **Image data:**
 - WH (4 Bytes each)
- **Total (W=H=12)**
 - $4 * (2H + 1 + WH) = 676$ bytes
 - Improvement by a factor 4.4



2c) Unpacking

Operations to pack and unpack a pixel

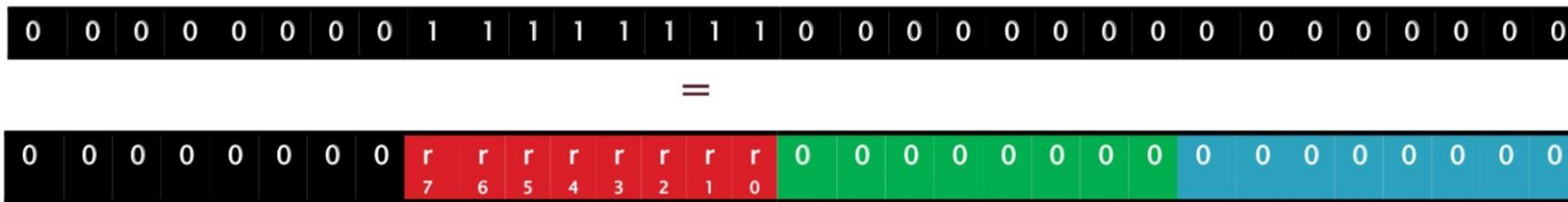
- ▶ **Right-shift operator ($A \gg B$)**; it shifts each bit of A to the right by B bits, and fills the least significant (rightmost) by zero.
- ▶ **Left-shift operator ($A \ll B$)**; it shifts each bit of A to the left by B bits, and fills the least significant (leftmost) by zero.
- ▶ **Bitwise logical operators:**
 - AND: &
 - OR : |
 - XOR: ^
 - Negation: ~



2c)



Bitwise Logical AND: &



Followed by a shift of 16 bits to the right: >> 16



```
// in Java
int c ...; // Pixel
int r = (c & 0x00ff0000) >> 16;
```

2c)

Shift of 16 bits to the right: $\gg 16$ Bitwise Logical AND: $\&$ 

=



```
// in Java
int c ...; // Pixel
int r = (c >> 16) & 0xff;
```

2d) Image Resolution

- Screen Size (Diagonal) = 6.3 inches
- Aspect Ratio = 19.5 : 9 $\rightarrow H = (19.5/9) \times W$
- Resolution = 1206 x 2622 pixels

According to Pythagora's Theorem:

$$W^2 + \{ (19.5/9) \times W \}^2 = (6.3)^2$$

Solving for W:

$$W = 2.64575 \text{ inches}$$

Therefore,

$$2.64575 \text{ inches} = 1206 \text{ pixels}$$

$$1 \text{ inch} \cong 456 \text{ pixels}$$



3a) Uniform Quantization

- Quantization: For an Image I , containing m different colors $C = \{C_1, C_2, \dots, C_m\}$, where $m \leq 2^{24}$ (For a 3 x 8 bit image), we have to replace the original colors with a set $C' = \{C_1, C_2, \dots, C_n\}$, with $n \ll m$ in such a way that we have minimal perceptible degradation in the resulting image.
- Uniform Quantization.
 - Each axis of the color space is divided into equal sized segments. (The number of these segments depends on storage requirements. For e.g. 8 for red, 8 for green and 4 for blue for 8 bit storage)
 - Each of the original colors is then mapped to the region which it falls in (256 for 8 bits).
 - Representative colors for each region is then the average of all the colors mapped to that region



3a) Median Cut Quanization

Source: **An Overview of Color Quantization Techniques** by Steven Segenchuk

https://web.cs.wpi.edu/~matt/courses/cs563/talks/color_quant/CQindex.html

- Median Cut Algorithm
 - Find the smallest box which contains all the colors in the image.
 - Sort the enclosed colors along the longest axis of the box.
 - Split the box into 2 regions at median of the sorted list.
 - Repeat the above process until the original color space has been divided into 256 regions.
 - The representative colors are found by averaging the colors in each box.



3b) (Random) Noise Dithering

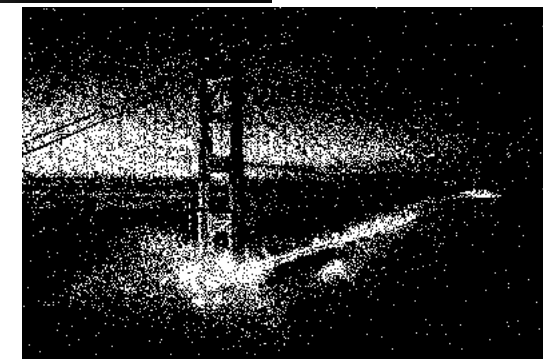
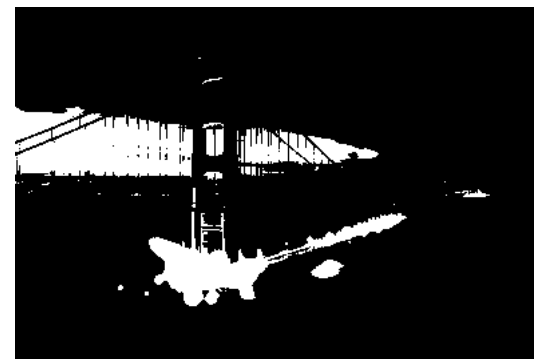
- Reduce effects of quantization (such as color banding) by adding uniformly distributed white noise (dither signal) to the input image prior to quantization.

Quantization (Without Dithering):

$$P(x, y) = Q(I(x, y)) = \text{floor}\left(\frac{I(x, y)}{256} 2^b\right)$$

Quantization (With Dithering):

$$P(x, y) = Q(I(x, y) + \text{noise}(x, y))$$



source: <https://surma.dev/things/ditherpunk/>